

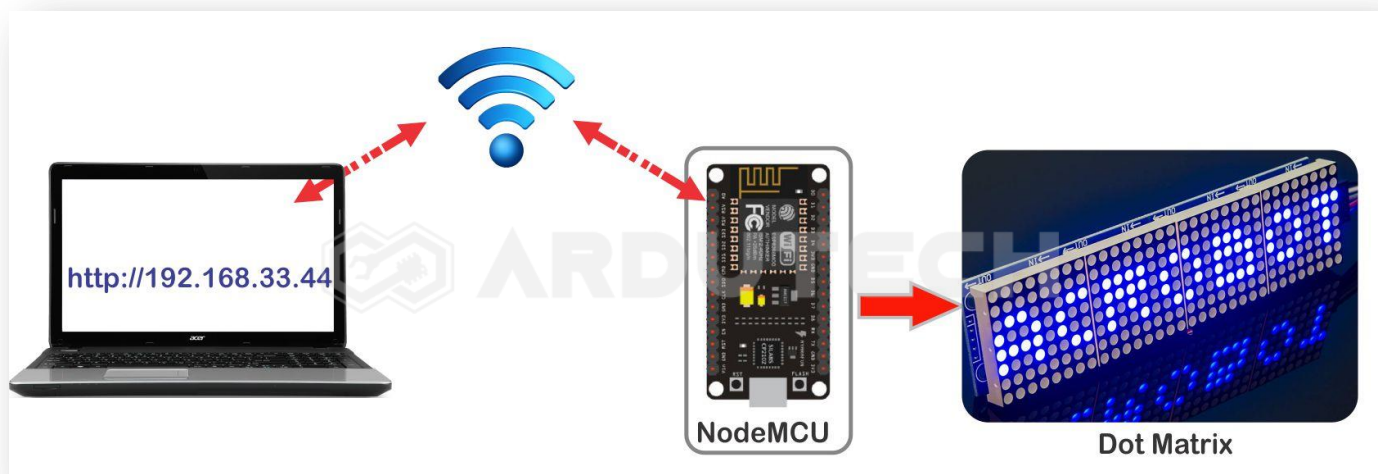
Menampilkan Text di Dot Matrix via WiFi

Deskripsi

Membuat proyek IoT (*Internet of Things*) untuk menampilkan text/tulisan di modul dot matrix dengan driver MAX7219 ukuran 8x32 secara *wireless* melalui jaringan WiFi dengan web page.

Cara Kerja

Modul WiFi NodeMCU V3 diset sebagai server kemudian meembuat halaman web (*web page*) yang dapat diakses melalui alamat IP (IP Address) . data berupa string kemudian dikirim dari web untuk diteruskan ke modul NodeMCU melalui jaringan WiFi yang terhubung dengan modul LED Matrix sehingga akan menampilkan text/tulisan tersebut.

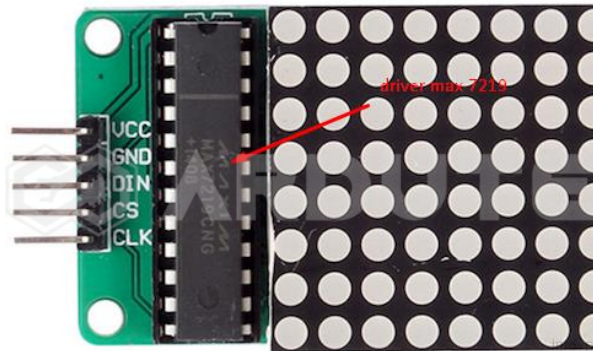


Modul dot matrix yang akan dipakai berukuran 8x32 yaitu terdiri dari 8 baris dan 32 kolom LED (4 modul LED 8x8) dengan driver max7219.



Spesifikasi modul LED Matrix 8x32 :

- Power supply : 5 VDC
- Interface : Serial SPI
- Controller : MAX7219



Pada modul LED Matrix 8x32 ini terdapat 5 pin yang dipakai untuk koneksi dengan mikrokontroler maupun NodeMCU.

Pin	Keterangan
VCC	Tegangan input 5VDC
GND	Ground
DIN	Data IN/MOSI
CS	SS
CLK	Clock/SCK

Kebutuhan Bahan

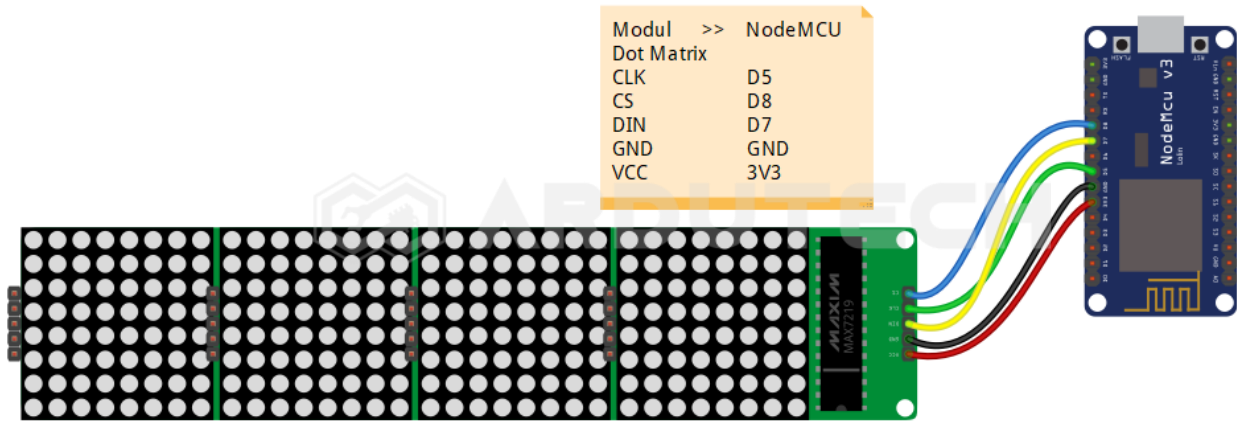
- NodeMCU V3
- Modul LED Matrix 8x32 Max7219
- Kabel konektor
- Kabel micro USB



Kebutuhan Software

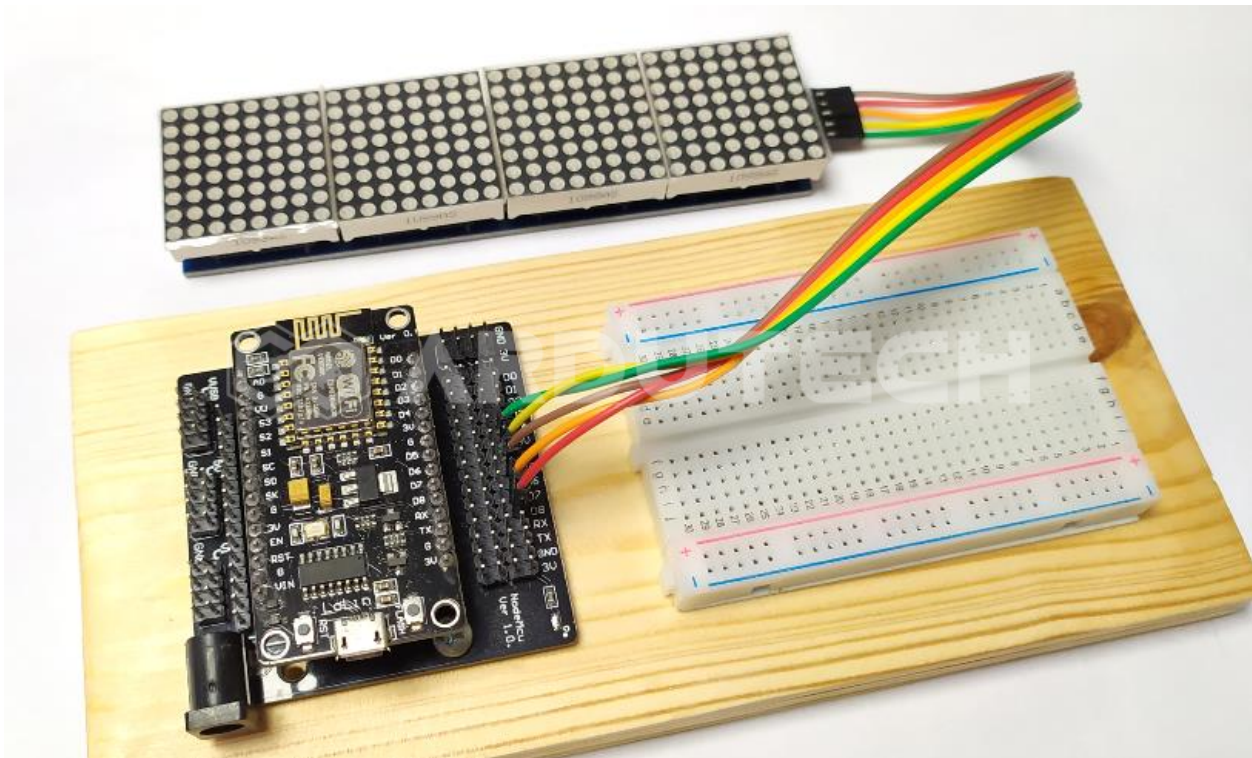
- Arduino IDE

Rangkaian/Skematik



Koneksi NodeMCU dengan Modul LED Matrix 8x32 Max7219 :

NodeMCU	Modul LED Matrix
D5	CLK
D7	DIN
D8	CS
GND	GND
Vin	VCC



Program/Source Code di Arduino IDE

Program pada proyek ini memerlukan library :

- MD_MAX72xx.h
- ESP8266WiFi.h
- SPI.h

Buka/jalankan Arduino IDE kemudian buat lembar kerja baru. Tulis kode program berikut.

```

/*****

* Project Menampilkan Text di Dot Matrix via WiFi
* Board   : NodeMCU ESP8266 V3
* Input    : web page
* Output   : LED Matrix 8x32
* 99 Proyek IoT
* www.ardutech.com

*****/

#include <ESP8266WiFi.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define PRINT_CALLBACK 0
#define DEBUG 0
#define LED_HEARTBEAT 0

#if DEBUG
#define PRINT(s, v) { Serial.print(F(s)); Serial.print(v); }
#define PRINTS(s)   { Serial.print(F(s)); }
#else
#define PRINT(s, v)
#define PRINTS(s)
#endif

#if LED_HEARTBEAT
#define HB_LED D2
#define HB_LED_TIME 500 // in milliseconds
#endif

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4

```

```

#define CLK_PIN    D5 // or SCK
#define DATA_PIN  D7 // or MOSI
#define CS_PIN     D8 // or SS

// SPI hardware interface
MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);
//---GANTI SESUAI DENGAN JARINGAN WIFI
//---HOTSPOT ANDA
const char* ssid = "ArdutechWiFi"; // Nama Hotspot/WiFi
const char* password = "12345678"; // Password

WiFiServer server(80);

const uint8_t MESG_SIZE = 255;
const uint8_t CHAR_SPACING = 1;
const uint8_t SCROLL_DELAY = 75;

char curMessage[MESG_SIZE];
char newMessage[MESG_SIZE];
bool newMessageAvailable = false;

const char WebResponse[] = "HTTP/1.1 200 OK\nContent-Type:
text/html\n\n";

const char WebPage[] =
"<!DOCTYPE html>" \
"<html>" \
"<head><meta name=\"viewport\" content=\"width=device-width, initial-
scale=1\">" \
"<title>Kontrol Running text</title>" \
"<style>" \

```



```
"html, body" \
"{ " \
"font-family: Helvetica; "\
"display: block;"\
"margin: 0px auto;"\
"text-align: center;"\
"background-color: #cad9c5;" \
}" \
"#container " \
"{ " \
"width: 100%;" \
"height: 100%;" \
"margin-left: 5px;" \
"margin-top: 20px;" \
"border: solid 2px;" \
"padding: 10px;" \
"background-color: #2dfa53;" \
}" \
"</style>"\
"<script>" \
"strLine = \"\";" \
"function SendText()" \
"{ " \
"  nocache = \"/&nocache=\" + Math.random() * 1000000;" \
"  var request = new XMLHttpRequest();" \
"    strLine = \"&MSG=\" +
document.getElementById(\"txt_form\").Message.value;" \
"  request.open(\"GET\", strLine + nocache, false);" \
"  request.send(null);" \
"}" \
"</script>" \
"</head>" \
```

```

"<body>" \
"<H1><b>Dot Matrix 7219 Module Control via WiFi</b></H1>" \

"<div id=\"container\">\" \
"<form id=\"txt_form\" name=\"frmText\">\" \
"<label>Message:<input          type=\"text\"          name=\"Message\" \
maxlength=\"255\"></label><br><br>\" \
"</form>\" \
"<br>\" \
"<input type=\"Send Text\" value=\"Send Text\" onclick=\"SendText()\">\" \
"</div>\" \
"</body>\" \
"</html>";

//-----
const char *err2Str(wl_status_t code)
{
    switch (code)
    {
        case WL_IDLE_STATUS:    return("IDLE");          break; // WiFi is in
process of changing between statuses
        case WL_NO_SSID_AVAIL:   return("NO_SSID_AVAIL"); break; // case
configured SSID cannot be reached
        case WL_CONNECTED:      return("CONNECTED");     break; // successful
connection is established
        case WL_CONNECT_FAILED: return("CONNECT_FAILED"); break; // password is
incorrect
        case WL_DISCONNECTED:    return("CONNECT_FAILED"); break; // module is
not configured in station mode
        default: return("??");
    }
}

//-----
uint8_t htoi(char c)
{

```

```
c = toupper(c);
if ((c >= '0') && (c <= '9')) return(c - '0');
if ((c >= 'A') && (c <= 'F')) return(c - 'A' + 0xa);
return(0);
}
//-----
boolean getText(char *szMesg, char *psz, uint8_t len)
{
    boolean isValid = false; // text received flag
    char *pStart, *pEnd;      // pointer to start and end of text
    pStart = strstr(szMesg, "&MSG=");

    if (pStart != NULL)
    {
        pStart += 6; // skip to start of data
        pEnd = strstr(pStart, "&");

        if (pEnd != NULL)
        {
            while (pStart != pEnd)
            {
                if ((*pStart == '%') && isdigit(*(pStart+1)))
                {
                    // replace %xx hex code with the ASCII character
                    char c = 0;
                    pStart++;
                    c += (htoi(*pStart++) << 4);
                    c += htoi(*pStart++);
                    *psz++ = c;
                }
                else
                {
                    *psz++ = *pStart++;
                }
            }
        }
    }
}
```



```
    }
    *psz = '\\0'; // terminate the string
    isValid = true;
}
}
return(isValid);
}
//-----
void handleWiFi(void)
{
    static enum { S_IDLE, S_WAIT_CONN, S_READ, S_EXTRACT, S_RESPONSE,
S_DISCONN } state = S_IDLE;
    static char szBuf[1024];
    static uint16_t idxBuf = 0;
    static WiFiClient client;
    static uint32_t timeStart;

    switch (state)
    {
    case S_IDLE: // initialize
        PRINTS("\\nS_IDLE");
        idxBuf = 0;
        state = S_WAIT_CONN;
        break;

    case S_WAIT_CONN: // waiting for connection
        {
            client = server.available();
            if (!client) break;
            if (!client.connected()) break;

#if DEBUG
```

```
    char szTxt[20];

    sprintf(szTxt, "%03d:%03d:%03d:%03d", client.remoteIP()[0],
client.remoteIP()[1], client.remoteIP()[2], client.remoteIP()[3]);

    PRINT("\nNew client @ ", szTxt);
#endif

    timeStart = millis();
    state = S_READ;
}
break;

case S_READ: // get the first line of data
    PRINTS("\nS_READ");
    while (client.available())
    {
        char c = client.read();
        if ((c == '\r') || (c == '\n'))
        {
            szBuf[idxBuf] = '\0';
            client.flush();
            PRINT("\nRecv: ", szBuf);
            state = S_EXTRACT;
        }
        else
            szBuf[idxBuf++] = (char)c;
    }
    if (millis() - timeStart > 1000)
    {
        PRINTS("\nWait timeout");
        state = S_DISCONN;
    }
    break;
```

```
case S_EXTRACT: // extract data
    PRINTS("\nS_EXTRACT");
    newMessageAvailable = getText(szBuf, newMessage, MESG_SIZE);
    PRINT("\nNew Msg: ", newMessage);
    state = S_RESPONSE;
    break;

case S_RESPONSE: // send the response to the client
    PRINTS("\nS_RESPONSE");
    client.print(WebResponse);
    client.print(WebPage);
    state = S_DISCONN;
    break;

case S_DISCONN: // disconnect client
    PRINTS("\nS_DISCONN");
    client.flush();
    client.stop();
    state = S_IDLE;
    break;

default: state = S_IDLE;
}
}

//-----

void scrollDataSink(uint8_t dev, MD_MAX72XX::transformType_t t, uint8_t col)
{
#ifdef PRINT_CALLBACK
    Serial.print("\n cb ");
    Serial.print(dev);
    Serial.print(' ');
```

```
Serial.print(t);
Serial.print(' ');
Serial.println(col);
#endif
}
//-----
uint8_t scrollDataSource(uint8_t dev, MD_MAX72XX::transformType_t t)
{
    static enum { S_IDLE, S_NEXT_CHAR, S_SHOW_CHAR, S_SHOW_SPACE } state =
    S_IDLE;
    static char *p;
    static uint16_t curLen, showLen;
    static uint8_t cBuf[8];
    uint8_t colData = 0;
    switch (state)
    {
    case S_IDLE:
        PRINTS("\nS_IDLE");
        p = curMessage;
        if (newMessageAvailable)
        {
            strcpy(curMessage, newMessage);
            newMessageAvailable = false;
        }
        state = S_NEXT_CHAR;
        break;

    case S_NEXT_CHAR:
        PRINTS("\nS_NEXT_CHAR");
        if (*p == '\0')
            state = S_IDLE;
        else
```

```
{
    showLen = mx.getChar(*p++, sizeof(cBuf) / sizeof(cBuf[0]), cBuf);
    curLen = 0;
    state = S_SHOW_CHAR;
}
break;

case S_SHOW_CHAR: // display the next part of the character
    PRINTS("\nS_SHOW_CHAR");
    colData = cBuf[curLen++];
    if (curLen < showLen)
        break;
    showLen = (*p != '\\0' ? CHAR_SPACING : (MAX_DEVICES*COL_SIZE)/2);
    curLen = 0;
    state = S_SHOW_SPACE;
    // fall through

case S_SHOW_SPACE:
    PRINT("\nS_ICSPACE: ", curLen);
    PRINT("/", showLen);
    curLen++;
    if (curLen == showLen)
        state = S_NEXT_CHAR;
    break;
default:
    state = S_IDLE;
}
return(colData);
}

//-----
void scrollText(void)
{
```

```
static uint32_t    prevTime = 0;

if (millis() - prevTime >= SCROLL_DELAY)
{
    mx.transform(MD_MAX72XX::TSL);
    prevTime = millis();
}
}

//=====

void setup()
{
    #if DEBUG
        Serial.begin(115200);
        PRINTS("\n[MD_MAX72XX WiFi Message Display]\nType a message for the
scrolling display from your internet browser");
    #endif

    #if LED_HEARTBEAT
        pinMode(HB_LED, OUTPUT);
        digitalWrite(HB_LED, LOW);
    #endif

    // Display initialization
    mx.begin();
    mx.setShiftDataInCallback(scrollDataSource);
    mx.setShiftDataOutCallback(scrollDataSink);

    curMessage[0] = newMessage[0] = '\0';
    PRINT("\nConnecting to ", ssid);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED)
    {
        PRINT("\n", err2Str(WiFi.status()));
    }
}
```

```
    delay(500);
}
PRINTS("\nWiFi connected");
server.begin();
PRINTS("\nServer started");
sprintf(curMessage, "%03d:%03d:%03d:%03d", WiFi.localIP()[0],
WiFi.localIP()[1], WiFi.localIP()[2], WiFi.localIP()[3]);
PRINT("\nAssigned IP ", curMessage);
}
//=====
void loop()
{
#if LED_HEARTBEAT
    static uint32_t timeLast = 0;
    if (millis() - timeLast >= HB_LED_TIME)
    {
        digitalWrite(HB_LED, digitalRead(HB_LED) == LOW ? HIGH : LOW);
        timeLast = millis();
    }
#endif
    handleWiFi();
    scrollText();
}
```

PERHATIKAN !

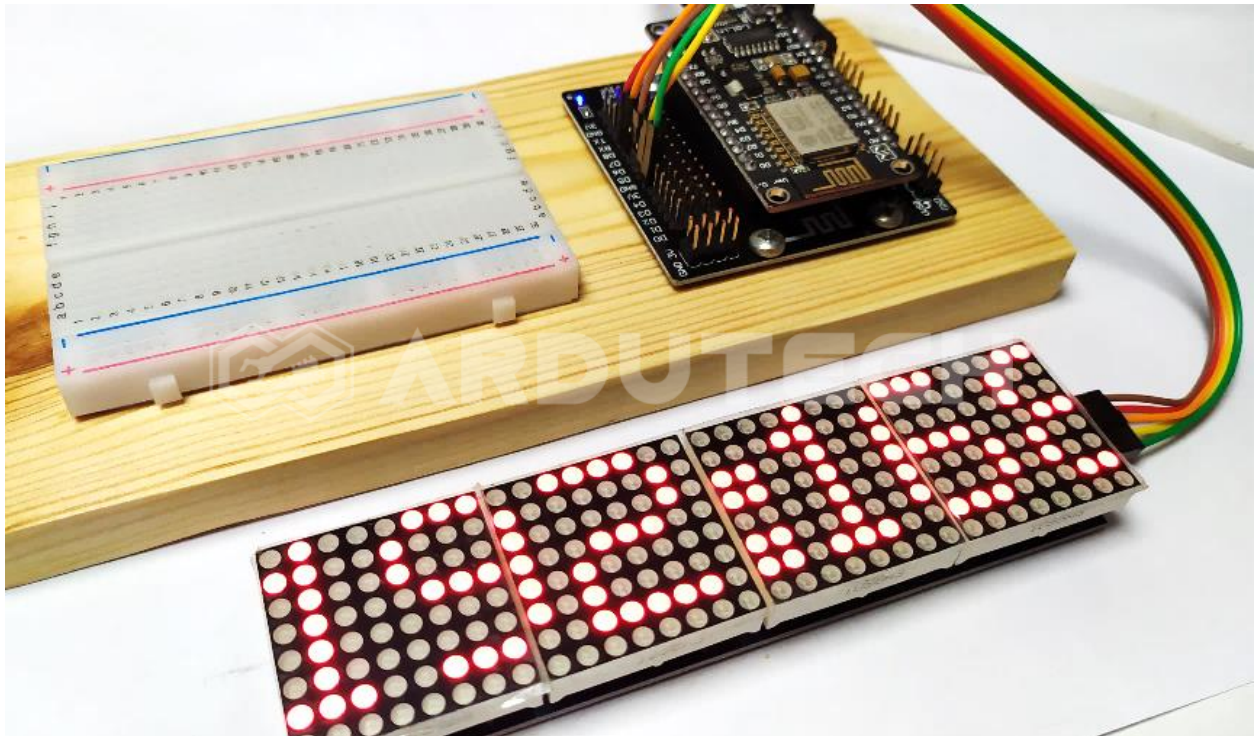
Ganti/sesuaikan variabel berikut :

- Nama jaringan WiFi/hotspot : **ssid**
- Password jaringan WiFi/hotspot : **password**

Simpan (Save) kemudian Upload. Pastikan tidak ada error, jika masih ada silakan cek penulisan dll kemudian perbaiki. (Program ini sudah diuji langsung dan sudah berjalan tanpa ada error)

Jalannya Alat

Setelah di Upload ke NodeMCU maka pada LED Matrix akan tampil :



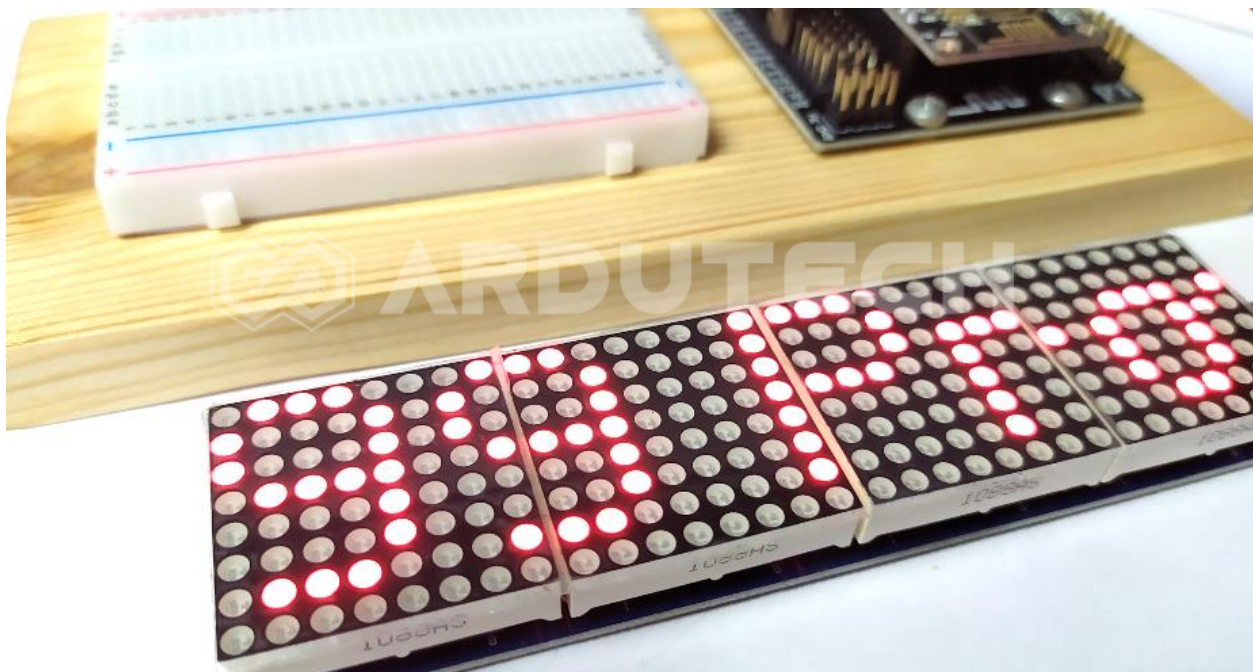
Selanjutnya pada browser silakan ketik alamat IP Address:



Ketik/tulis pada bagian “Message” , misalnya : “Ardutech 99 Project IoT” kemudian tekan tombol “Send Text”



Hasilnya akan tampil di LED Matrix yang terhubung dengan NodeMCU :



Selamat berkarya , semoga lancar dan bermanfaat.

Ardutech – “Sahabat Inovasi Anda”